Computer Vision 실습 개요 소개

실습순서

1. OpenCV

1. OpenCV Basic
2. OpenCV를 이용한 전통적인 컴퓨터 비전 기본 프로젝트 실습

2. Deep Learning

1. Face Detection (얼굴검출, Media Pipe)
2. Face Recognition (얼굴인식 및 유사도 검사, InsightFace)
3. OCR (광학문자인식)
4. Object Detection (객체검출, YOLO)
5. 동영상 및 웹캠 실시간 객체검출 (YOLO)

실습순서

3. Yolo Custom Dataset

1. YOLO person-vehicle object detection (Yolo 학습형태의 데이터셋 구조)
2. YOLO Bolt-Nut-Washer object detection (직접 수집한 이미지에 바운딩박스 라벨링, Roboflow)
3. YOLO 객체 검출 성능평가
4. YOLO 산업용 비전검사 - 동영상 웹캠 부품 수량검사 (부품 수량 검사 : Bolt-Nut-Washer)

4. Yolo Pose / Segmentation / 모델 서빙

1. YOLO Pose [양팔들기 감지, squat 횟수 카운팅]
2. YOLO Segmentation (인스턴스 세그멘테이션 : 이미지 개체 분할) [danger-zone : 작업자 위험구간 진입 감지]
3. FastAPI 모델 서빙 [부품수량검사 모델 inference server]

OpenCV를 이용한 전통적인 컴퓨터 비전

컴퓨터가 이미지나 영상을 사람의 눈처럼 인식하고 처리할 수 있도록 돕는 오픈소스 컴퓨터 비전 및 이미지 처리 라이브러리

1. 주요 특징

실시간 영상 처리 특화: C와 C++ 등으로 작성되어 속도가 매우 빠르며, GPU 가속(CUDA, OpenCL 등)을 지원
다양한 언어 및 운영체제 지원: Python, C++, Java 등의 언어를 지원하며, Windows, Linux, macOS, Android, iOS 등 다양한 플랫폼에서 동작함
방대한 알고리즘 보유: 이미지 필터링, 이진화, 외곽선 검출, 객체 추적, 3차원 비전 등 2,500개 이상의 최적화된 알고리즘을 포함하고 있음
AI 및 딥러닝 연동: TensorFlow, PyTorch, YOLO 등 딥러닝 프레임워크와 결합해 실시간 객체 검출 및 AI 모델 구동이 가능함
오픈소스 라이선스: 무료로 제공되며 상업적 및 비상업적 목적 모두 사용 가능 (현재 Apache License 2.0 적용)

2. 주요 활용 분야

자율주행: 차선 인식, 보행자 및 차량 탐지, 표지판 인식
로보틱스: 로봇 비전, 주변 환경 인식, 물체 조작 및 내비게이션
스마트 공장: 제품 불량 검사 및 자동 품질 검사 (머신비전)
보안 및 출입 관리: CCTV 영상 분석, 얼굴 인식 출입 시스템
의료 및 엔터테인먼트: 의료 영상 분석, AR 필터 및 모션 캡처

3. 파이썬에서 시작하기 (기본 설치)

파이썬 환경에서는 pip 명령어로 간단히 설치하여 cv2 모듈로 사용 가능

```
pip install opencv-python
```

딥러닝을 이용한 컴퓨터 비전 (Vision AI)

채용공고에서도 OCR·문서 파싱·VLM을 결합한 기술과 YOLO 기반 객체 검출·배포 역량이 요구됨

비전 언어 모델(VLM, Vision Language Model): 이미지나 영상 등의 시각적 데이터와 텍스트를 동시에 이해하고 처리하는 멀티모달 AI 모델

컴퓨터비전 기술

기술	핵심 라이브러리	활용 서비스
OCR 문자 인식	EasyOCR, PaddleOCR	영수증, 문서, 송장 인식
얼굴 검출	OpenCV, MediaPipe	얼굴 위치 검출, 모자이크
얼굴 인식	InsightFace	출입 인증, 등록 얼굴 비교
객체 검출	YOLO	CCTV, 제품 검출
객체 추적	YOLO + ByteTrack	사람·차량 추적
자세 추정	MediaPipe, YOLO Pose	운동 자세 분석
이미지 분할 추천	YOLO Segmentation	불량 검출, 객체 영역 추출

AI 엔지니어 취업 포트폴리오로는 딥러닝기반 비전 AI가 더 많은 실무 역량을 보여줄 수 있음.

실제 주니어 컴퓨터비전 엔지니어 채용공고에서도 딥러닝 객체 검출, Segmentation, PyTorch, OpenCV, Git, Docker 등의 역량을 함께 제시

실습 01. 웹캠 얼굴 검출 및 모자이크 (MediaPipe + OpenCV)

핵심 기술: MediaPipe Face Detection, OpenCV, ROI, Mosaic



1. 사용사례
웹캠이나 CCTV에서 얼굴을 자동으로 검출하고 모자이크 처리하는 기술.
영상 개인정보 보호, 방송 영상 편집, 거리 촬영 영상의 얼굴 비식별화 등에 활용 가능

이번 실습에서는 사전학습된 MediaPipe 얼굴 검출 모델로 얼굴 위치를 찾고, OpenCV로 모자이크 처리

2. 실습 목표
- MediaPipe 사전학습 얼굴 검출 모델 활용
- OpenCV로 웹캠 영상 읽기
- 얼굴 Bounding Box 좌표 추출
- ROI(Region of Interest)로 얼굴 영역 분리
- 이미지 축소·확대로 모자이크 구현
- 실시간으로 여러 사람의 얼굴 처리

MediaPipe는 BlazeFace라는 경량 딥러닝 얼굴 검출 모델을 제공. 이번 실습에서는 별도 모델 학습 없이 사전학습 모델을 사용
구글 MediaPipe 가이드 Face Detection 가이드

위 경로에서 BlazeFace 경량 모델 .tflite 을 직접 다운받아 사용하거나. 링크주소를 복사하여 urllib.request 로 다운로드한 후 사용

필요한 모듈 설치
```
pip install mediapipe opencv-python
```

실습 01-1. 동영상 얼굴 검출 및 모자이크 처리와 영상 저장

핵심 기술: MediaPipe Face Detection, OpenCV, VideoCapture, VideoWriter, Mosaic

1. 실습 목표

- 동영상 파일에서 프레임 단위 이미지 추출
- MediaPipe 사전학습 모델로 얼굴 검출
- 검출된 얼굴 영역에 모자이크 적용
- 모자이크 처리된 영상을 MP4 파일로 저장

필요한 모듈 설치

```
pip install mediapipe opencv-python
```

실습 01-2. 동영상 얼굴 검출 및 캐릭터 이미지 씌우기

핵심 기술: MediaPipe Face Detection, OpenCV, VideoCapture, VideoWriter, Mosaic

1. 실습 목표

- 동영상 파일에서 프레임 단위 이미지 추출
- MediaPipe 사전학습 모델로 얼굴 검출
- 검출된 얼굴 영역에 캐릭터 이미지 덮기
- 모자이크 처리된 영상을 MP4 파일로 저장

필요한 모듈 설치

```
pip install mediapipe opencv-python
```

실습 02. 얼굴 인식 및 유사도 비교 (InsightFace)

핵심 기술: Face Recognition, Face Embedding, Cosine Similarity

1. 실습 개요

서로 다른 두 사진에 등장하는 얼굴이 같은 사람인지 비교하는 실습.
이전 실습에서는 MediaPipe로 얼굴의 위치만 검출. 이번에는 InsightFace의 사전학습 딥러닝 모델로 얼굴의 특징을 추출하고 유사도를 계산.

구분	이전 실습	이번 실습
기술	Face Detection	Face Recognition
목적	얼굴 위치 찾기	두 얼굴의 유사도 비교
결과	Bounding Box	Embedding, 유사도 점수
라이브러리	MediaPipe	InsightFace [**InsightFace: 얼굴 검출, 인식, 분석을 위해 개발된 오픈소스 딥러닝 라이브러리 및 툴킷**]

주요 기능

- 얼굴 검출 (Face Detection): 이미지나 영상에서 사람의 얼굴 영역과 위치(Bouding Box), 랜드마크를 찾아냄
- 얼굴 인식 및 임베딩 (Face Recognition): 얼굴을 512차원 등의 벡터(임베딩)로 변환해 동일인 여부를 판별. ArcFace 같은 첨단 손실 함수를 사용해 정확도가 높음
- 속성 분석: 얼굴 정보를 바탕으로 나이, 성별, 머리 방향(Pose) 등을 추정.
- 얼굴 바꾸기 (Face Swap): inswapper 같은 모델을 활용해 사진이나 영상 속 인물의 얼굴을 다른 사람의 얼굴로 자연스럽게 합성

특징

- 오픈소스: MIT 라이선스를 따르며 연구 및 상용 목적으로 활용 가능
- 파이썬 지원: pip install insightface 명령어로 쉽게 설치할 수 있으며, PyTorch나 ONNX Runtime 등과 연동해 사용

실습 02. 얼굴 인식 및 유사도 비교 (InsightFace)

모듈 설치 [설치중 numba 라는 모듈이 numpy 버전과 충돌날 수 있으나 특정 연산을 빠르게 실행하도록 돕는 모듈이므로 필수 아님]

```
pip install -U insightface opencv-python numpy
```

InsightFace 2.1은 ONNX Runtime을 기본 설치하여 사용함.

ONNX Runtime(ORT)는 MS 개발한 고성능 머신러닝 모델 추론(Inference) 및 학습 가속화 엔진. (AI 서비스 배포의 성능향상을 위해 요즘 많이 사용됨-요구기술 중 하나)

ONNX(Open Neural Network Exchange)

서로 다른 머신러닝·딥러닝 프레임워크 간에 모델을 자유롭게 공유하고 실행할 수 있도록 설계된 공통 개방형 표준 포맷

주요 특징

프레임워크 상호 운용성: PyTorch, TensorFlow, Scikit-learn 등 다양한 프레임워크에서 만든 모델을 ONNX(.onnx) 표준 형식으로 변환하여 실행할 수 있음
다양한 하드웨어 가속 지원: CPU, CUDA, TensorRT, DirectML, OpenVINO 등 하드웨어별 실행 공급자(Execution Provider)를 통해 최적의 속도를 냄
크로스 플랫폼 및 멀티 언어: Windows, Linux, macOS, Android, iOS 등 다양한 OS를 지원하며, Python, C++, C#, Java 등 여러 프로그래밍 언어를 지원
모델 최적화: 그래프 최적화(Graph Optimization), 노드 융합(Node Fusion), 상수로 값 미리 계산(Constant Folding) 등을 통해 추론 속도를 극대화

주요 활용 사례

온디바이스(Edge) AI: 모바일 앱, 엣지 디바이스, 브라우저(WebAssembly) 등 리소스가 제한된 환경에서 가볍고 빠르게 AI 모델을 구동
서버 및 클라우드 배포: 파이썬 등으로 학습한 무거운 모델을 운영(Production) 환경에서 C++나 C# 기반의 고성능 서비스로 배포할 때 주로 사용

실습 03. EasyOCR을 이용한 한글·영문 문자 인식

핵심 기술: EasyOCR, Text Detection, Text Recognition

1. 실습 개요
사진 속 한글과 영어를 자동으로 찾아 텍스트로 추출하는 실습
OCR(Optical Character Recognition)은 영수증 분석, 문서 디지털화, 간판 인식, 택배 송장 처리 등에 활용
이번 실습에서는 사전학습된 EasyOCR 모델을 사용하므로 별도 딥러닝 학습은 필요하지 않음

2. 실습 목표
- 사전학습 OCR 모델 사용
- 이미지에서 한글·영문 자동 인식
- 인식된 글자의 위치(Bounding Box) 추출
- 문자 인식 신뢰도(Confidence) 확인
- 검출된 문자 영역을 이미지에 표시

모듈 설치

```
pip install easyocr opencv-python
```

EasyOCR은 내부적으로 PyTorch를 사용. 설치 문제가 발생하면 PyTorch를 먼저 설치

실습용 이미지 준비
한글과 영어가 함께 있는 간판, 안내문 또는 문서 사진을 준비

실습 04. 객체 검출의 이해와 YOLO 이미지 객체 검출

객체 검출(Object Detection)의 두 가지 접근 방식인 Single-Stage와 Two-Stage를 비교하고, YOLO26 사전학습 모델을 이용하여 이미지에서 사람, 자동차, 동물 등의 객체를 검출

1. 학습 목표
- Image Classification과 Object Detection의 차이 이해
- Single-Stage와 Two-Stage 객체 검출 구조 차이 이해
- 사전학습 YOLO11 or YOLO26 모델 활용
- Bounding Box, Class, Confidence 정보 추출
- OpenCV로 검출 결과 표시 및 저장
- Confidence 임계값 변경에 따른 결과 비교

구분	이미지 분류 (Image Classification)	객체 탐지 (Object Detection)
핵심 질문	"이 이미지에 무엇(What)이 담겨 있는가?"	"이 이미지의 어디(Where)에 무엇(What)이 있는가?"
출력 결과	이미지 전체의 단일 클래스(Label)와 확률값 (예: 강아지 (0.95))	객체별 클래스 이름 + 위치 좌표(Bounding Box) 목록
객체 수	보통 이미지 내 하나의 주된 객체 식별	이미지 내 다수(여러 개)의 객체 동시 인식
주요 알고리즘	CNN 기반 모델 (ResNet, GoogLeNet 등)	YOLO, SSD, R-CNN 계열 등
평가 지표	정확도(Accuracy), Top-k Accuracy 등	IoU(Intersection over Union), mAP(mean Average Precision) 등

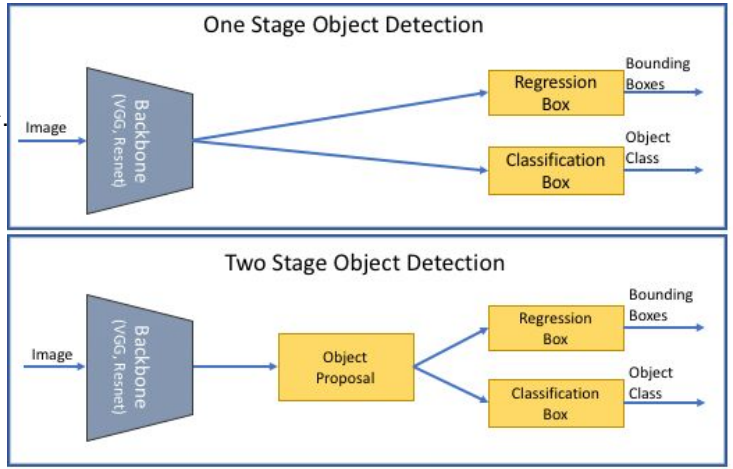
객체 탐지(Object Detection) 시스템

이미지나 비디오 내의 객체를 식별하고 위치를 추정하는 알고리즘 또는 모델을 의미. 객체 탐지에서 detector는 특정 객체가 이미지에 존재하는지 여부를 판단하고, 그 객체의 위치를 나타내는 bounding box를 생성하는 역할함. 이 과정은 크게 두 가지로 나누어짐

Classification (분류): 객체가 어떤 종류인지를 예측하는 단계. 예) 이미지 내의 어떤 객체가 사람인지, 고양이인지, 자동차인지 분류하는 작업을 수행
Localization (위치 추정): 객체가 이미지 안에서 어느 위치에 존재하는지를 판단. 각 객체에 대해 bounding box를 생성. 해당 객체가 포함된 영역을 사각형으로 표시

객체 탐지 모델은 이 두 가지 작업을 동시에 수행하는 방식으로, 분류와 위치 추정을 모두 처리함.
객체 탐지 모델이 생성한 bounding box는 객체의 위치를 정의하며, 모델은 각 박스가 어떤 객체에 해당하는지를 예측합니다.

2-Stage vs. 1-Stage Detector
객체 탐지(Object Detection) 모델은 크게 1-stage detector와 2-stage detector로 구분.
이 두 방식은 객체 탐지의 처리 방식과 속도, 성능에서 차이를 보이며 각각의 장단점이 있음.



2-Stage Detector
먼저 이미지 내에서 객체가 있을 법한 후보 영역을 추출한 후, 그 영역에 대해 분류를 수행.
정확도는 높지만 처리 시간이 상대적으로 걸림. 대표모델 : Faster R-CNN

1-Stage Detector
anchor box와 CNN을 통한 동시 처리 방식으로, 이미지를 한 번에 처리하여 객체를 탐지.
속도가 빠르지만 정확도는 상대적으로 낮을 수 있음. 대표모델 : YOLO

객체 탐지에서 detector는 객체를 찾아내고, 그 위치와 종류를 예측하는 핵심적인 역할을 수행

실습 04. 객체 검출의 이해와 YOLO 이미지 객체 검출

실습 구성

YOLO 사전학습 모델을 활용한 이미지 객체 검출
- 라이브러리: ultralytics, opencv-python
- 모델: YOLO11 Nano (yolo11n.pt) or YOLO26 Nano (yolo26n.pt)
- 실습내용: 이미지에서 사람, 자동차, 동물 등을 검출하고 Bounding Box, 클래스명, 신뢰도 표시
- 별도 모델 학습이나 Colab GPU 불필요

Ultralytics 울트라리틱스

실시간 컴퓨터 비전 AI 모델인 YOLO(You Only Look Once)를 쉽고 빠르게 개발, 학습, 배포할 수 있도록 지원하는 대표적인 오픈소스 라이브러리 및 AI 기업

YOLO 모델 지원: YOLOv5, YOLOv8, YOLO11 등 최신 YOLO 계열 모델의 공식 구현 및 패키지를 제공

다양한 비전 작업 처리
- 객체 탐지 (Object Detection): 이미지 속 물체의 위치와 종류 파악
- 분할 (Segmentation): 물체의 정확한 외곽선(픽셀 단위) 추출
- 분류 (Classification): 이미지 전체가 어떤 클래스인지 판별
- 포즈 추정 (Pose Estimation): 사람의 관절 등 키포인트 추적
- 추적 (Tracking): 영상 내 객체 연속 추적

모듈 설치

```
pip install ultralytics opencv-python
```

이번 실습에서는 YOLO11 Nano(yolo11n.pt) or 26모델을 사용.
작은 모델이므로 CPU에서도 진행하기에 적합하고, 학습된 가중치는 최초 사용 시 자동으로 다운로드 됨.

실습 04. 객체 검출의 이해와 YOLO 이미지 객체 검출

실습용 이미지 준비

bus.jpg : 버스와 여러 사람을 동시에 검출
zidane.jpg : 사람 객체가 겹쳐 있을 때 검출 결과를 비교
boats.jpg : 작은 객체가 많이 등장하는 이미지에서 검출 결과를 관찰

사전학습 YOLO 모델 불러오기

model = YOLO("yolo11n.pt") / model = YOLO("yolo26n.pt")
yolo11n.pt : YOLO11 Nano 사전학습 모델
최초 실행 시 모델 파일 자동 다운로드
COCO 데이터셋의 80개 객체 클래스 검출
클래스 종류 소개 : https://tech.amikelive.com/node-718/what-object-categories-labels-are-in-coco-dataset/#google_vignette

YOLO 모델 버전 종류

yolo11n.pt : YOLO11 Nano | yolo11s.pt : YOLO11 Small | yolo11m.pt : YOLO11 Medium
yolo11l.pt : YOLO11 Large | yolo11x.pt : YOLO11 Extra Large
m (중간) 버전까지는 CPU에서 어떻게든 실습 가능하지만, l(대형)과 x(초대형) 버전은 GPU 환경에서만 실습 가능

버전 13까지 배포 후 14~25까지 건너뛴 후 26버전으로 네이밍. (SOTA를 달성한 모델이 2026년도에 나와서)
SOTA(State-of-the-Art) 모델 : 특정 인공지능(AI)이나 기계학습 분야의 특정 작업에 대해 정해진 평가 기준(벤치마크) 안에서 현시점에 가장 높은 성능(최고 점수, 최저 오류율 등)을 기록한 모델

기본적이고 검증된 프로덕션 용도 : yolo11
최신 SOTA 및 최고 속도/성능 실험 : yolo26
용량이 너무 크면 실습이 너무 오래 걸리므로, 가급적 m 이하로 실습진행.

학습하지 않은 객체를 검출하고 싶다면?

1. Custom Training

새로운 객체 이미지와 정답 라벨로 YOLO 재학습

2. 다른 사전학습 모델 사용

원하는 클래스가 포함된 모델 선택

3. Open-Vocabulary Detection

텍스트로 검출할 객체를 지정하는 모델 활용

실습 05. YOLO 모델을 새로운 데이터셋으로 학습시키기

이미 객체에 대한 라벨링이 된 Ultralytics의 다른 데이터셋을 로드하여 학습 시키기
COCO와 VOC 외에도 목적별로 특화된 여러 데이터셋들이 존재함.

Ultralytics의 데이터셋

COCO:	일반 객체 80종	범용 객체 탐지	훈련:118,287장	검증:5,000장	테스트:20,288장
Pascal VOC:	일반 객체 20종	학습 실습, 비교 실험	훈련:11,530장	검증:Train에 통합	테스트:4,952장
COCO8	이미지 8장	코드 동작 확인	훈련: 4장	검증: 4장	테스트: 없음
COCO128	이미지 128장	빠른 학습 테스트	훈련: 128장	검증: 128장(동일)	테스트: 없음
Objects365	객체 365종	다양한 객체 탐지	훈련: 1,742,289장	검증: 80,000장	테스트: 없음
VisDrone	드론 시점 객체 10종	항공 촬영, 작은 객체	훈련: 6,471장	검증: 548장	테스트: 1,610장
SKU-110K	진열대의 상품 탐지	매장 상품 탐지	훈련: 8,219장	검증: 588장	테스트: 2,936장
TT100K	다양한 교통표지판	도로 표지판 탐지	훈련: 6,105장	검증: 7,641장	테스트: 3,071장

데이터셋으로 학습하기

```
model.train(
    data="coco8.yaml,  # Ultralytics 라이브러리 내부 패키지에 기본 설정 파일이 내장되어 있고, 데이터셋이 처음 실행될 때 자동으로 다운로드
    epochs=10,
    imgsz=640,
    batch=4
)
```

실습 05. YOLO 모델을 새로운 데이터셋으로 학습시키기

데이터셋 YAML 파일 명

- COCO: coco.yaml
- Pascal VOC: VOC.yaml
- COCO8: coco8.yaml
- COCO128: coco128.yaml
- Objects365: Objects365.yaml
- VisDrone: VisDrone.yaml

YOLO 학습을 위한 data.yaml 파일 예시

yaml

```
# 데이터셋 루트 경로 (선택 사항이나 보통 상대/절대 경로 기준점 역할)
path: /content/dataset  # 예: Colab이나 로컬 프로젝트 경로

# 학습, 검증, 테스트 이미지 폴더 경로 (path 기준 또는 절대 경로)
train: train/images     # 학습용 이미지 경로 (예: /content/dataset/train/images)
val: valid/images       # 검증용 이미지 경로 (예: /content/dataset/valid/images)
test: test/images       # (선택 사항) 테스트용 이미지 경로

# 클래스 개수 (Number of Classes)
nc: 2

# 클래스 이름 목록 (0부터 시작하는 인덱스와 정확히 매칭)
names: ['cat', 'dog']
```

YAML 파일이란

야믈 파일(YAML 또는 YML 파일)은 사람이 쉽게 읽고 쓸 수 있도록 설계된 데이터 직렬화 양식. 이름은 "YAML Ain't Markup Language"의 줄임말로 마크업 언어가 아님을 뜻하는 재귀적 약자

실습 06. Open-Vocabulary Detection 방식

YOLO-World 같은 모델은 기존 YOLO와 달리 텍스트로 찾고 싶은 객체를 지정할 수 있음.
YOLO-World : 텐센트 AI랩(Tencent AILab) 컴퓨터 비전 센터에서 개발한 실시간 개방-어휘(Open-vocabulary) 객체 탐지 모델. Ultralytics YOLOv8을 기반으로 구축했음.

기존의 객체 탐지 모델과 달리 미리 정의된 특정 클래스(Category)만 감지하는 것이 아니라, 텍스트 프롬프트를 활용해 훈련 시 보지 못했던 새로운 사물이나 개념도 실시간으로 찾아낼 수 있는 것이 특징

비전-언어 모델링 결합: 비전(이미지)과 언어(텍스트) 데이터를 함께 학습하여, 텍스트 설명만으로 객체를 인식
오프라인 어휘(Prompt-then-detect) 방식: 사용자가 원하는 클래스를 코드(set_classes() 등)로 자유롭게 지정하여 탐지할 수 있음

YOLOv8s-world, m, l, x 및 v2 버전 등의 모델이 제공
객체 추적 지원: 단일 이미지 탐지뿐만 아니라 비디오 소스에서 움직이는 여러 객체를 실시간으로 추적하는 기능도 지원
가이드 문서 : https://docs.ultralytics.com/ko/models/yolo-world

```
model = YOLOWorld("yolov8s-world.pt")
```

검출할 객체를 텍스트로 지정 (helmet(안전모), safety vest(안전조끼), person(사람))

```
model.set_classes(["helmet", "safety vest", "person"])
```

다만 텍스트로 클래스를 지정할 수 있다고 해서 모든 물체를 정확하게 검출하는 것은 아님.
실무에서 Open-Vocabulary 객체 탐지(OVD) 모델은 제한적이지만 데이터 라벨링 자동화와 초기 프로토타이핑 단계를 중심으로 빠르게 도입되고 있음.
다만, 실제 서비스 시스템(프로덕션)에 탑재하는 최종 모델로는 여전히 Ultralytics YOLO 시리즈(YOLOv8, YOLO11 등) 같은 고정 어휘 기반의 탐지 모델이 압도적으로 많이 사용

실습 07. YOLO + OpenCV 동영상 및 웹캠 실시간 객체 검출

핵심 기술: YOLO11, OpenCV, VideoCapture, VideoWriter, Object Detection

1. 실습 개요
이전 실습에서는 이미지 한 장에서 객체를 검출했음.
이번에는 동영상이나 웹캠에서 프레임을 계속 읽어 객체를 검출하고, Bounding Box를 표시한 결과를 영상 파일로 저장.
실제 활용 사례로는 CCTV의 사람·차량 검출, 실시간 안전모 착용 감지, 무인 매장 모니터링 등이 있음.

2. 학습 목표
- OpenCV로 동영상 및 웹캠 영상 읽기
- YOLO 모델로 프레임별 객체 검출
- Bounding Box, 클래스명, 신뢰도 표시
- 현재 프레임의 객체 개수 확인
- 검출 결과를 MP4 동영상으로 저장

모듈설치
pip install ultralytics opencv-python

YOLO 결과 객체의 plot() 함수로 Bounding Box를 자동으로 표시
YOLO 모델이 객체를 검출하면 내부적으로 경계 상자(Bounding Box)의 좌표, 예측한 클래스(이름), 신뢰도(Confidence) 점수를 데이터 형태로 가지고 있음.
• result.plot()은 이 복잡한 데이터들을 원본 이미지 위에 보기 좋게 그려서(시각화하여) 새로운 이미지(픽셀 데이터)로 반환하는 함수
• 개발자가 일일이 OpenCV의 cv2.rectangle()이나 cv2.putText()를 써서 상자와 글씨를 그릴 필요 없이, 단 한 줄로 알아서 색상별 상자, 클래스 이름, 확률(%)을 채워 넣은 프레임을 만들어 줌.

실습 08. YOLO Object Tracking — 움직이는 객체 추적하기

핵심 기술: YOLO11, ByteTrack, Object ID, OpenCV

1. 실습 개요
이전 실습에서는 동영상의 각 프레임에서 객체를 검출했음. 이번에는 같은 사람이나 자동차가 이동하더라도 동일한 ID를 유지하도록 추적
예를 들어 CCTV에 세 명이 등장하면 사람마다 ID 1, ID 2, ID 3을 부여하고, 각 객체가 움직여도 가능한 한 동일한 ID를 유지

Tracking 단계
① 동영상 프레임 입력 : OpenCV VideoCapture

② YOLO 객체 검출 : Bounding Box + Class + Confidence

③ ByteTrack 객체 연결 : 이전 프레임과 현재 프레임의 객체 매칭

④ Tracking ID 부여 : 동일 객체에 같은 ID 유지 시도

⑤ 객체 이동 경로 표시 : 중심 좌표를 연결하여 궤적 시각화

ByteTrack은 객체의 위치와 움직임 등을 이용해 프레임 간 검출 결과를 연결하는 추적 알고리즘임.
낮은 신뢰도의 검출 정보도 연결에 활용하여 일시적인 검출 누락을 줄이도록 설계되어 있음.

모듈 설치
pip install ultralytics opencv-python numpy

YOLO Custom Dataset Fine-tuning

실습 환경: GPU · PyTorch · YOLO26
핵심 기술: Transfer Learning, Fine-tuning, Custom Dataset, Annotation, YOLO Training
처음부터 이미지 수집과 Annotation을 진행하면 학습 데이터 준비에 많은 시간을 사용하고, 정작 Fine-tuning 원리를 이해하기 어려울 수 있음.

욜로 파인튜닝 실습 순서 4단계

1) yolo 학습형태의 데이터셋 구조를 가진 데이터셋을 직접 만들지 않고 캐글에서 다운로드한 후 파인튜닝하기
person vehicle(사전학습모델에 없는 클래스명) 만 검출하고 인식하도록 사전학습 모델을 파인튜닝
(사전학습모델의 바운딩박스 찾는 가중치값을 재 사용하되 classification을 새로 학습)
--> .yaml 파일의 역할과 그로인한 학습데이터셋 구조 이해
--> YOLO 데이터셋 구조와 Annotation 형식 이해 (YOLO 학습에는 '이미지와 해당 객체의 위치를 표시한 Annotation(정답 라벨)'이 필요)
--> 사전학습 모델과 Fine-tuning의 차이 이해
--> GPU 활성화 및 확인
--> model.train()으로 Custom Dataset 학습
--> 학습 완료 후 best.pt 모델 사용
--> 학습 모델을 다운로드하여 재사용

2) 직접 수집한 이미지들을 가지고 yolo 데이터셋 구조로 annotation(bbox 좌표+라벨명)을 만들고 이를 학습한 best.pt 모델 만들기
비전검사(제품 불량 판별, 기준치 미달, 용접상태 등)[프로젝트 주제], 등. 특정 도메인용 욜로모델 만들기
(예. DIY가구 부속품 포장할때 너트2개, 볼트2개, 패킹4개 등.. 객체인식 및 수량)

3) mAP 등 객체인식 성능평가 지표.

4) 파인튜닝한 모델로 영상속 객체 검출.

실습 01. YOLO person vehicle object detection

person vehicle(사전학습모델이 없는 클래스명) 만 검출하고 인식하도록 사전학습 모델을 파인튜닝

1. 실습 개요

지금까지는 COCO 데이터셋으로 사전 학습된 yolo11n.pt는 80개의 클래스를 구분했음. 이 클래스에는 car, bus, truck, bike 등 탈것(vehicle)를 세분화 하여 분류해줌

우리가 만들어 볼 모델은 이전 모델에는 없는 단순히 사람(person)과 탈것(vehicle) 2개의 클래스만 구분하는 객체인식모델임.

이를 위해 사람과 탈것이 함께있는 여러 이미지를 준비하고 이 이미지들의 각 객체별 boundingbox와 분류번호(클래스 구분)가 작성된 annotation 문서를 준비할것.

처음부터 이미지 수집과 Annotation을 진행하면 학습 데이터 준비에 많은 시간을 사용하고, 정작 Fine-tuning 원리를 이해하기 어려울 수 있어서

이미 만들어진 YOLO 데이터셋을 다운로드 하여 구조를 살펴보고 yolo custom 모델을 만들어보기

실습 목표

GPU 설정 및 Kaggle 데이터셋 다운로드

이미지와 Annotation .txt 파일 확인

YOLO 정규화 좌표와 Bounding Box 시각화

data.yaml 구조 및 역할 이해

사전학습 yolo11n.pt를 2개 클래스로 Fine-tuning

best.pt 생성 및 학습 결과 확인

best.pt 파일을 불러와서 객체 검출

실습 01. YOLO person vehicle object detection

[참고]
YOLO11과 YOLO26 모두 기본 제공되는 공식 사전 학습 모델(Pre-trained Model)을 그대로 사용할 경우, 똑같이 COCO 데이터셋 기준의 80가지 클래스를 탐지
두 모델의 탐지 클래스는 다음과 같이 구성
• 동일한 80개 범주: 사람(Person), 자동차(Car), 자전거(Bicycle) 를 포함해 가방, 우산, 컵, 의자 등 일상에서 흔히 볼 수 있는 80가지 사물이 지정되어 있음.
• 클래스 번호 매핑: 내부 클래스 인덱스(0번부터 79번까지)와 탐지 대상 목록(0: person, 1: bicycle, 2: car...) 역시 동일한 COCO 표준 형식을 따름.

YOLOE-26
앞서 언급된 YOLO26 라인업의 확장 모델인 YOLOE-26(오픈 어휘록 모델)은 고정된 80개 클래스에 갇히지 않고, 사용자가 텍스트 프롬프트로 "빨간 컵 찾아줘"라고 입력하면 80개 클래스에 포함되지 않은 새로운 물체도 유연하게 탐지할 수 있음.
사업용 서비스에 이용할 경우 유료

데이터셋 링크
https://www.kaggle.com/datasets/haseebhsb/yolo-person-vehicle-detection-dataset-annotated?utm_source=chatgpt.com

데이터셋 주요 정보
- 전체 이미지 2,000장
- Training 1,600장
- Validation 400장
- Bounding Box Annotation 4,041개
- 클래스 2개 (person, vehicle)
- Annotation 형식 YOLO 형식의 파일별 .txt

실습 02. YOLO Bolt Nut Washer Object Detection

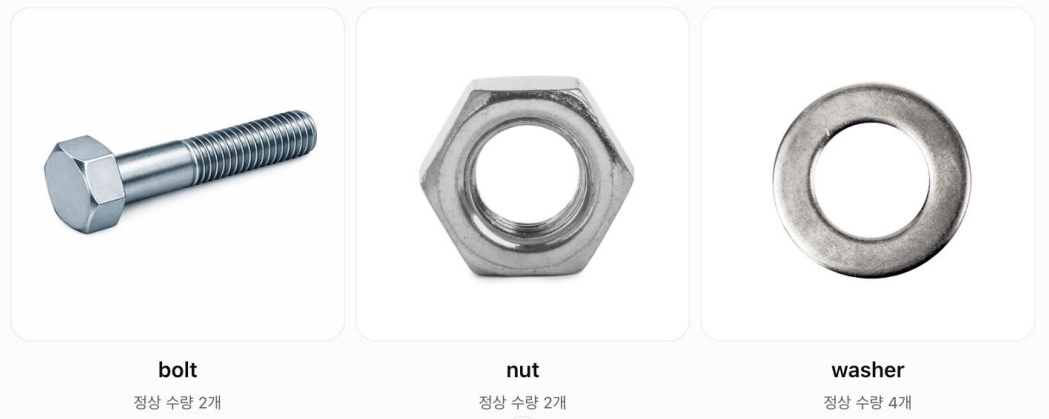
DIY 가구 부속품 포장 수량 검사 AI 만들기

직접 수집한 이미지에 Bounding Box를 라벨링하고 YOLO 데이터셋을 구축하여, 특정 산업용 검사 모델 제작
DIY가구 부속품 포장할때 너트2개, 볼트2개, 와셔2개 객체인식 및 수량

실습 환경: GPU · YOLO26

준비물: 직접 Annotation을 완료한 bolt, nut, washer YOLO 데이터셋

- YOLO: 사진 속 볼트·너트·와셔의 종류와 위치를 검출
- Python: 클래스별 수량을 계산하여 각각 2개인지 검사
- 최종 결과: 모두 정확히 2개면 PASS, 하나라도 부족하거나 초과하면 FAIL



실습 02. Roboflow로 YOLO Annotation 만들기

이미지 수집을 완료했다면 이제 볼트(bolt), 너트(nut), 와셔(washer)의 위치를 직접 표시하고 클래스명을 지정하는 작업을 진행

```
class_id center_x center_y width height
```

```
0 0.25 0.30 0.12 0.20
0 0.72 0.30 0.12 0.20
1 0.30 0.60 0.10 0.10
1 0.70 0.60 0.10 0.10
2 0.20 0.80 0.08 0.08
2 0.40 0.80 0.08 0.08
2 0.60 0.80 0.08 0.08
2 0.80 0.80 0.08 0.08
```

Roboflow Annotation 작업 순서

1. 프로젝트 생성

Roboflow 접속 → 로그인 → 새 프로젝트 생성

- Project Name: DIY-Furniture-Parts
- Project Type: Object Detection
- Annotation Type: Bounding Box

무료 Public 플랜에서는 데이터셋이 공개될 수 있으므로 직접 촬영한 비공개 산업 현장 사진이라면 공개 여부를 먼저 확인

2. 수집한 이미지 업로드

Upload Data 메뉴에서 JPG 또는 PNG 파일을 드래그하여 업로드. 처음에는 10장 정도만 등록해서 라벨링 기준을 확인한 후 나머지 이미지를 업로드해도 좋음.

3. Annotation 화면 열기

Annotate 또는 Dataset 화면에서 이미지를 선택하여 라벨링 도구를 열기. 오른쪽 도구 모음에서 사각형 아이콘(Bounding Box)을 선택

4. Bounding Box 지정

마우스로 객체의 왼쪽 위부터 오른쪽 아래까지 드래그. 박스를 만든 다음 클래스 선택 메뉴에서 bolt, nut, washer 중 하나를 지정

5. 작업 저장 및 다음 이미지 처리

한 이미지의 모든 객체를 표시한 후 저장·완료 처리하고 다음 이미지로 이동. UI 버전에 따라 버튼 명칭은 다를 수 있음.
이미지 10장의 라벨을 확인하고 나서 전체 데이터셋으로 확대

실습 03. YOLO 객체 검출 모델 성능평가

사용 모델: 직접 학습한 best.pt
사용 데이터: Annotation이 포함된 Validation / Test 데이터

학습할 성능평가 지표

IoU	예측 Bounding Box와 정답 Box의 겹침 정도
Precision	검출했다고 판단한 객체 중 실제 정답의 비율
Recall	실제 객체 중 모델이 검출한 비율
F1-Score	Precision과 Recall의 조화평균
mAP@50	IoU 0.5 기준 평균 검출 성능
mAP@50:95	IoU 0.5~0.95 기준 평균 성능
Confusion Matrix	클래스별 정답·오분류 분석

mAP@50 추가 설명 : 예측 박스와 정답 박스가 50% 이상 겹치기만 하면 올바르게 검출(True Positive)된 것으로 관대하게 판정
mAP@50:95 추가 설명 : IoU 임계값을 0.50부터 0.95까지 0.05 단위로 10개(0.50, 0.55, 0.60, ..., 0.95) 적용하여 구한 mAP들의 평균값
단순 객체 감지뿐만 아니라 위치(Localization)를 얼마나 정밀하게 예측하는지까지 엄격하게 평가하므로, 모델의 실질적인 완성도를 평가함.

평가 항목 학생이 확인할 내용

IoU Bounding Box가 얼마나 정확하게 겹치는가?
Precision 잘못 검출한 객체가 많은가?
Recall 놓친 부품이 많은가?
mAP50 / mAP50-95 전체적인 객체 검출 성능이 좋은가?
Confusion Matrix 어떤 부품을 자주 혼동하는가?
포장 검사 정확도 실제 정상/불량 판정을 잘하는가?

실습 04. YOLO 산업용 비전검사 - 동영상 웹캠 부품 수량검사

사용 모델: bolt, nut, washer를 학습한 best.pt (기존 모델 사용, 재학습 없음)

1. 완성할 검사 시스템
정상 포장의 기준 - Bolt: 2개, Nut: 2개, Washer: 2개
화면 표시: Bounding Box + 클래스별 수량 + 우상단 녹색 [PASS] / 빨간색 [FAIL]

실제 결과 화면에서는 모든 부품에 Bounding Box를 표시하고, 화면 오른쪽 위에 큰 글씨로 다음과 같이 출력
- 수량이 모두 정확히 2개이면 초록색 [PASS]
- 한 종류라도 부족하거나 초과하면 빨간색 [FAIL]

동영상 파일을 처리해 결과 MP4로 저장하거나, Streemlit, Gradio를 이용해 브라우저 웹캠 화면을 연속적으로 검사하는 두 가지 방식을 사용

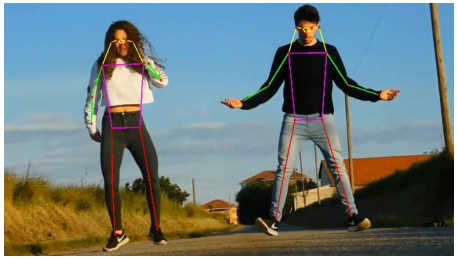
2. 실행순서
Fine-tuning한 best.pt 업로드
공통 객체 검출 및 수량 검사 코드 실행
검사할 동영상 업로드 → 결과 MP4 재생
선택적으로 Streemlit, Gradio 웹캠 실행

YOLO Pose Estimation & Segmentation

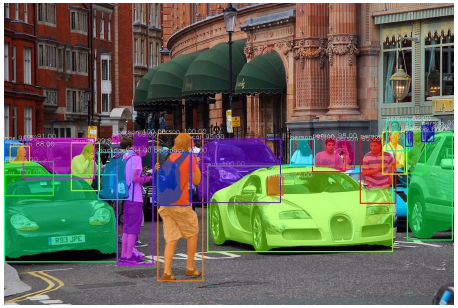
Object Detection은 객체를 사각형(Bounding Box)으로 검출하는 방법이었음. 이번에는 YOLO가 객체의 자세와 형태까지 분석하는 방법을 실습

1. Pose와 Segmentation은 무엇인가?

① Pose Estimation — 사람의 자세 추정
사람의 어깨, 팔꿈치, 손목, 무릎 등 관절 위치를 찾아 점과 선으로 연결.
활용: 작업자 자세 분석, 운동 자세 교정, 낙상 감지
모델 : `yolo26n-pose.pt`



② Instance Segmentation — 객체 영역 분할
사각형이 아니라 객체의 실제 윤곽에 해당하는 픽셀 영역을 찾아 마스크로 표시
활용: 제품 표면 영역 분석, 결함 영역 분할, 부품 면적 측정
모델 : `yolo26n-seg.pt`

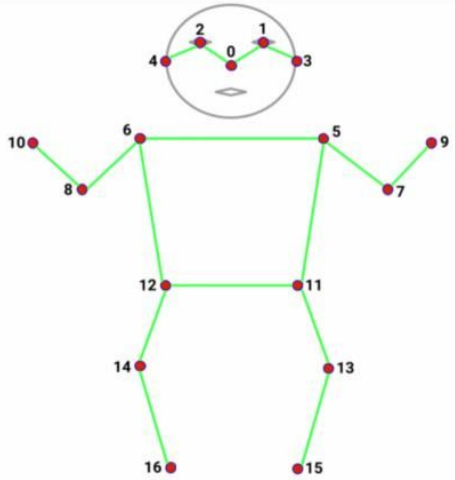


실습 01. YOLO Pose Estimation

(1) ARMS UP 양손들기

양쪽 손목이 각각의 귀보다 위에 있으면 ARMS UP, 그렇지 않으면 NORMAL을 표시





Index	Keypoint
0	nose
1	left_eye
2	right_eye
3	left_ear
4	right_ear
5	left_shoulder
6	right_shoulder
7	left_elbow
8	right_elbow
9	left_wrist
10	right_wrist
11	left_hip
12	right_hip
13	left_knee
14	right_knee
15	left_ankle
16	right_ankle

(2) Squat 스쿼트

무릎 각도를 기준으로 UP과 DOWN 상태를 구별하고, DOWN 상태에서 다시 UP 상태로 바뀔 때만 횟수를 증가



엉덩이(Hip): 11번 또는 12번
무릎(Knee): 13번 또는 14번
발목(Ankle): 15번 또는 16번

실습 02. YOLO Segmentation

YOLO Segmentation으로 작업자 위험구역 침입 감지

작업자가 위험구역에 들어오면 DANGER, 그렇지 않으면 SAFE를 표시



기존 Object Detection은 Bounding Box를 사용하지만, Segmentation은 사람의 실제 윤곽에 가까운 픽셀 마스크를 제공

최종실습. FastAPI로 YOLO 객체 검출 API 만들기 (모델 서빙)

사용 모델: 직접 Fine-tuning한 bolt, nut, washer 모델 (best.pt)
이번 실습은 지금까지 만든 DIY 가구 부속품 검사 모델을 다른 프로그램에서도 사용할 수 있는 REST API 서비스로 만드는 것

별도의 웹사이트를 개발하지 않고, FastAPI가 자동 제공하는 Swagger UI((http://127.0.0.1:8000/docs)에서 이미지를 업로드하여 검사

실습 목표
- FastAPI로 객체 검출 서버 생성
- 이미지 파일을 HTTP POST 요청으로 업로드
- YOLO best.pt로 부품 검출
- bolt, nut, washer 수량 계산
- 정상 기준인 2개씩과 비교하여 PASS / FAIL 반환
- JSON 응답 및 Swagger UI 사용법 이해

FastAPI의 UploadFile로 업로드한 이미지를 읽고, Ultralytics의 result.boxes에서 클래스·신뢰도·Bounding Box를 얻는 방식

모듈 설치

```
pip install fastapi uvicorn python-multipart ultralytics opencv-python
```

python-multipart: FastAPI가 이미지 업로드 요청을 처리하기 위해 필요한 라이브러리